



Universidade Federal do Cariri
Ciência da Computação
Juazeiro do Norte, CE
2025

Computação Gráfica

Material de Apoio VII: Algoritmos de Seleção

Monitores: Felipe Coêlho e Luma Araujo
Profª Responsável: Luana Batista

Sumário

1.	Introdução	3
2.	Conceito Geral	4
2.1.	Aplicações Práticas	4
3.	Seleção de Ponto	5
3.1.	Definição Básica	5
3.2.	Problema de Precisão	5
3.3.	Região de Tolerância	6
3.4.	Implementação Prática	6
4.	Seleção de Linha	7
4.1.	Abordagem Simples	7
4.2.	Abordagem Eficiente	8
4.2.1.	Ideia Principal	8
4.2.2.	Codificação dos Vértices (<i>bitcodes</i>)	8
4.2.3.	Casos Triviais	9
4.2.4.	Casos Não Triviais	9
5.	Seleção de Área	11
5.1.	Ideia Principal	11
5.2.	Casos Triviais	11

5.3.	Casos Não Triviais (Cálculo Da Interseção)	12
5.4.	Casos Especiais	13

1 - Introdução

Em ambientes gráficos interativos, não basta apenas desenhar objetos na tela — é fundamental permitir que o usuário **interaja com esses objetos** (Figura 1), selecionando-os para mover, editar ou aplicar transformações.

Para isso, usamos os **algoritmos de seleção**, que são um conjunto de técnicas que verificam se um objeto na tela foi escolhido a partir de uma posição de clique $m(x, y)$. Esses algoritmos funcionam para objetos básicos como:

- **Ponto**
- **Linha (segmento de reta)**
- **Área** (polígono)

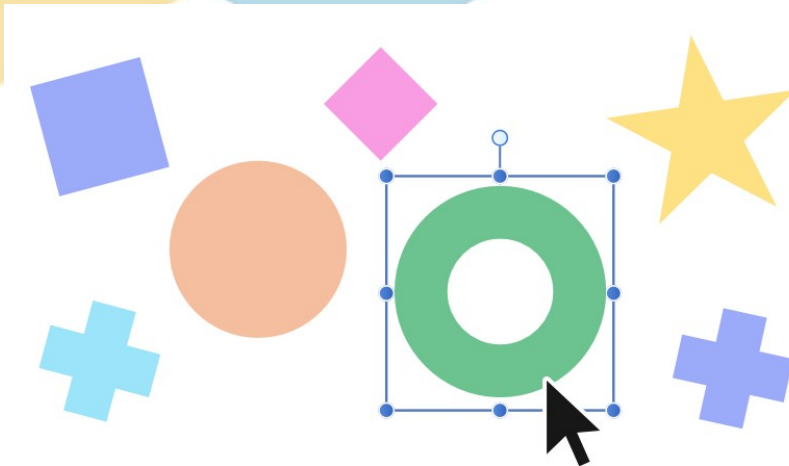


Figura 1 - Algoritmos de seleção lidam com questões como tolerância de clique, precisão numérica e eficiência computacional, sendo essenciais em aplicações que exigem **interatividade em tempo real**.

2 - Conceito Geral

Os **algoritmos de seleção (*pick*)** têm como objetivo determinar se um objeto gráfico foi ou não selecionado a partir de uma posição do mouse $m(x, y)$.

2.1 Aplicações Práticas

Os algoritmos de seleção aparecem em vários contextos de Computação Gráfica e áreas relacionadas, como:

- **Softwares de desenho e modelagem:** Permitem selecionar pontos, linhas ou áreas para edição.
- **Jogos digitais:** Detectam quando o jogador clica ou interage com objetos na tela.

- **Interfaces gráficas (GUIs):** Indicam se botões, ícones ou regiões foram selecionados.
- **Sistemas de CAD/CAE:** Selecionam elementos geométricos para aplicar transformações e operações de engenharia.
- **Aplicações de Realidade Virtual/Aumentada:** Reconhecem de objetos virtuais a partir de gestos ou apontamentos.

3 - Seleção de Ponto

A **seleção de ponto** é o caso mais simples entre os algoritmos de seleção. Ela consiste em verificar se o ponto do clique do mouse $m(x, y)$ coincide com a posição de um ponto gráfico $p(x, y)$ previamente desenhado na tela.

3.1 Definição Básica

- Um ponto é considerado selecionado se:

$$m(x, y) = p(x, y)$$

Na prática, isso significa que o clique do usuário ocorreu exatamente sobre o ponto.

3.2 Problema de Precisão

Na maioria das vezes, o clique do usuário não será exatamente na coordenada do ponto devido a:

- Limitações da resolução da tela.
- Dificuldade de posicionar o cursor em um único pixel.
- Pequenas diferenças numéricas no cálculo das coordenadas.

Por isso, utiliza-se o conceito de **tolerância de seleção**.

3.3 Região de Tolerância

Define-se uma pequena área ao redor do ponto, geralmente um **quadrado** (Figura 2) **ou círculo de raio r** em torno de $p(x, y)$.

- O ponto é considerado selecionado se o clique $m(x, y)$ estiver dentro dessa região.

3.4 Implementação Prática

- **Entrada:** coordenadas do clique (m_x, m_y) e do ponto gráfico (p_x, p_y) .
- **Processo:** verificar se (m_x, m_y) está dentro de uma região de tolerância (t) .
- **Saída:** verdadeiro/falso → ponto selecionado ou não.

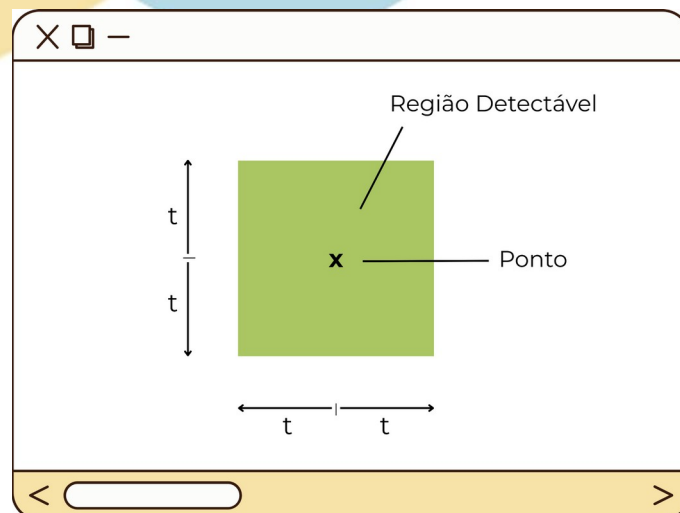


Figura 2 - Exemplificação gráfica da região quadrada detectável de um ponto qualquer, data uma tolerância t .

4 - Seleção de Linha

A **seleção de linha** é mais complexa que a seleção de ponto, uma vez que envolve verificar se um clique próximo a um segmento de reta deve ser considerado uma seleção.

4.1 Abordagem Simples

A ideia mais direta seria calcular a **distância do ponto de clique** $m(x, y)$ **até a reta que contém o segmento**.

- Fórmula da distância ponto-reta:

$$d = \frac{|ax_m + by_m + c|}{\sqrt{a^2 + b^2}}$$

onde $ax + by + c = 0$ é a equação da reta.

- O clique é aceito se $d \leq tol$, onde **tol** é a tolerância definida (em pixels).

Problema:

Esse cálculo exige operações de raiz quadrada e verificações adicionais para garantir que o ponto está dentro do **segmento** (e não em uma extensão infinita da reta), tornando o algoritmo **computacionalmente caro** quando se trata de muitos objetos.

4.2 Abordagem Eficiente

O algoritmo de recorte de **Cohen-Sutherland**, usado para determinar se um segmento intercepta uma região retangular de recorte, torna-se uma solução mais prática. Este método verifica se o segmento de reta a ser selecionado intercepta o retângulo de tolerância alinhado aos eixos de coordenadas, tendo como centro o ponto $m(x, y)$.

4.2.1 Ideia Principal

- Cria-se um **retângulo de tolerância** centrado no ponto de clique $m(x, y)$, com largura e altura definidas pelo parâmetro de tolerância (**tol**).
- Verifica-se se o segmento intercepta esse retângulo.
- Se sim $\rightarrow$ a linha foi selecionada.

4.2.2 Codificação dos Vértices (*bitcodes*)

Cada extremidade do segmento recebe um código de **4 bits** com base em sua posição relativa ao retângulo (Figura 3):

- **Esquerda:** $px < mx - tol$
- **Direita:** $px > mx + tol$
- **Abaixo:** $py < my - tol$
- **Acima:** $py > my + tol$

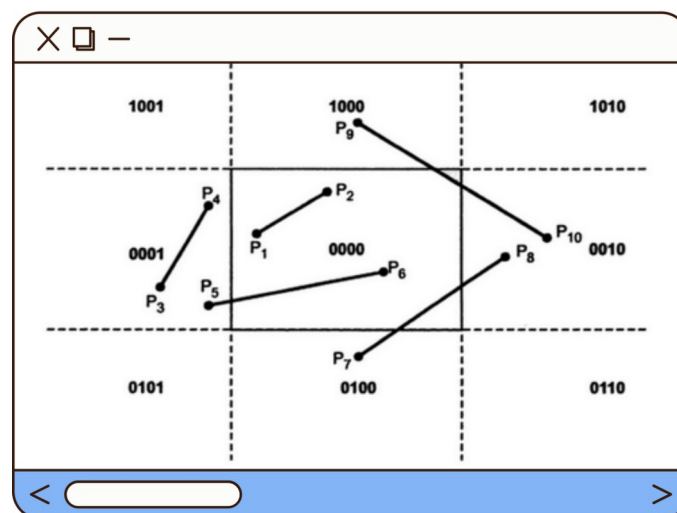


Figura 3 - Exemplificação gráfica da codificação de vértices.

4.2.3 Casos Triviais

- Código = 0000 → o ponto está dentro (intercepta).
- Segmentos totalmente fora (de mesmo lado) → eliminados via operação **AND** **bit** **a** **bit.**

4.2.4 Casos Não Triviais

Se o resultado da operação **AND** for **0000**, ainda pode haver interseção. Nesse caso, aplica-se a regra de movimentar o vértice inicial até a borda do retângulo:

● **Se à esquerda:**

$$x_0 = x_{min}, \quad y_0 = y_0 + \frac{(x_{min} - x_0)(y_1 - y_0)}{(x_1 - x_0)}$$

● **Se à direita:**

$$x_0 = x_{max}, \quad y_0 = y_0 + \frac{(x_{max} - x_0)(y_1 - y_0)}{(x_1 - x_0)}$$

● **Se abaixo:**

$$y_0 = y_{min}, \quad x_0 = x_0 + \frac{(y_{min} - y_0)(x_1 - x_0)}{(y_1 - y_0)}$$

● **Se acima:**

$$y_0 = y_{max}, \quad x_0 = x_0 + \frac{(y_{max} - y_0)(x_1 - x_0)}{(y_1 - y_0)}$$

Depois de reposicionar, o vértice é **recodificado** e volta-se à análise dos casos triviais. Esse processo é repetido até que o segmento seja aceito ou

rejeitado.

Observação

Para garantir a precisão no cálculo, determine primeiro a abscisa e ordenada da interseção e apenas depois defina a coordenada correspondente à borda do retângulo.

5 - Seleção de Área

A seleção de áreas é usada quando queremos determinar se um ponto $m(x, y)$, originado por um clique do usuário, está **dentro ou fora de um polígono**. O método mais comum para essa tarefa é o **algoritmo do tiro (ray casting)**, também conhecido como método "shoot", muito usado em Computação Gráfica e em Geometria Computacional.

5.1 Ideia Principal

O algoritmo funciona da seguinte forma:

1. A partir do ponto $m(x, y)$, dispara-se uma **semirreta horizontal** no sentido positivo do eixo x .
2. Conta-se o número de vezes que essa semirreta cruza as arestas do polígono.
 - Se o número de interseções for **ímpar** → o ponto está **dentro** do polígono.
 - Se for **par** → o ponto está **fora**.

Esse princípio é conhecido como **teorema do número de interseções**.

5.2 Casos Triviais

Nem sempre é necessário calcular a interseção da semirreta com todas as arestas. Alguns casos podem ser eliminados de imediato, tornando o algoritmo mais eficiente:

- **Arestas totalmente acima do ponto:**
 $(p1.y > y) \wedge (p2.y > y) \rightarrow$ não há interseção.
- **Arestas totalmente abaixo do ponto:**
 $(p1.y < y) \wedge (p2.y < y) \rightarrow$ não há interseção.
- **Arestas totalmente à esquerda do ponto:**
 $(p1.x < x) \wedge (p2.x < x) \rightarrow$ não há interseção.
- **Arestas que cruzam a altura do ponto e estão à direita dele:**
 $(p1.x > x \wedge p2.x > x) \wedge ((p1.y > y \wedge p2.y < y) \vee (p1.y < y \wedge p2.y > y)) \rightarrow$ podem gerar interseção e precisam ser analisadas.

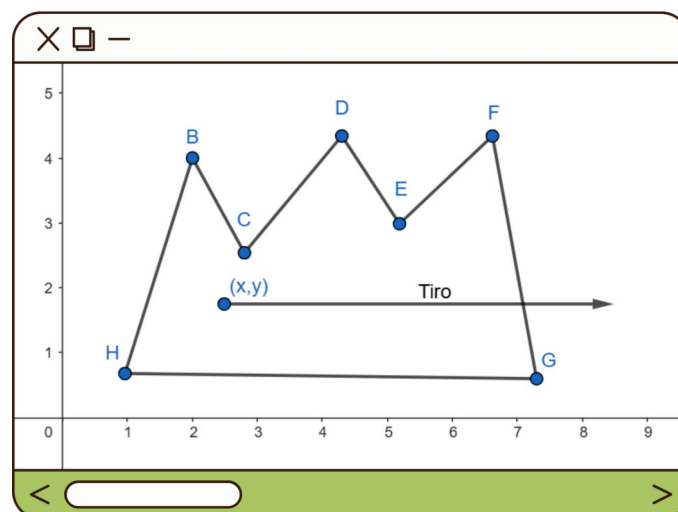


Figura 4 - Exemplificação gráfica do algoritmo do tiro, dado o ponto de clique nas coordenadas (x, y) e o polígono exemplificado.

5.3 Casos Não Triviais (Cálculo Da Interseção)

Quando a aresta não se enquadra em nenhum dos descartes triviais, é necessário calcular a coordenada x da interseção da aresta com a semirreta:

$$x_i = p1.x + \frac{(y - p1.y)(p2.x - p1.x)}{(p2.y - p1.y)}$$

- Se $x_i > x$, significa que a semirreta cruzou a aresta → conta-se uma interseção.
- Caso contrário, não há interseção.

5.4 Casos Especiais

Existem duas situações que exigem atenção para evitar resultados incorretos:

1. Interseção no vértice do polígono

- O tiro pode passar exatamente por um vértice, que pertence a duas arestas.
- **Regra:** conta-se a interseção apenas quando a outra extremidade da aresta está **acima** do ponto (ou, em algumas implementações, apenas quando está abaixo).

2. Arestas horizontais na altura do ponto

- Quando $p1.y = p2.y = y$, a aresta é **horizontal** e está na mesma altura do tiro.
- **Regra: ignorar essa aresta**, pois sua inclusão pode levar a erros de contagem.

Limitação:

- Funciona bem para **polígonos convexos e côncavos fechados**.
- Não funciona corretamente em **polilinhas abertas** ou em formas degeneradas (com interseções próprias).